

Análisis Exploratorio de Datos (EDA)

En esta sección del notebook se realiza un análisis exploratorio para el dataset de detección de obstáculos en interiores conformado, abarcando los siguientes pasos:

Estructura del EDA:

- **Paso 1:** Unificación de Datasets base
- **Paso 2:** Análisis Técnico del Dataset (resolución, integridad, duplicados, estadísticas de píxeles)
- **Paso 3:** Distribución de Clases (conteos, co-ocurrencia, comparación train/val/test)
- **Paso 4:** Geometría de Bounding Boxes (tamaño, aspect ratio, heatmap espacial, anchors optimizados)
- **Paso 5:** Complejidad de Escena (densidad de objetos, oclusión/IoU)
- **Paso 6:** Inspección Visual Cualitativa (muestreo aleatorio, detección de anomalías)

```
In [1]: # Auto-reload: recarga módulos automáticamente al detectar cambios en
%load_ext autoreload
%autoreload 2

# Librerías
import os
import sys
import json
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import cv2
import random
import hashlib
import yaml
from pycocotools.coco import COCO
from sklearn.cluster import KMeans
from pathlib import Path
import zipfile

# Rutas base
base_dir = "/Users/admin/Documents/TFM_UNIR/01_ING_DATOS"

# Añadir path de utilidades
src_path = os.path.join(base_dir, "src")
if src_path not in sys.path:
    sys.path.insert(0, src_path)
from utils_eda import *

# Directorio de salida para guardar reportes
output_dir = os.path.join(base_dir, "Reports", "EDA_Dataset_0IDC_origi
```

Paso 1: Descomprimir Dataset y Mapear Imágenes

En este paso se toma el Dataset **Indoor Object Detection Custom -versión original-** en archivo ZIP con formato YOLO26, se descomprime y se prepara para su análisis.

 Este dataset Versión Original cuenta con 1074 fotos, a partir de 5 sub-

dataset:

- **Dataset_Custom_1:** 512 Fotos tomadas desde ESP32S3 + CAM OV5640 a resolución FRAMSIZE_QVGA 320x240 pixeles [baja iluminación - noche]
- **Dataset_Custom_2:** 265 Fotos tomadas desde ESP32S3 + CAM OV5640 a resolución FRAMSIZE_SVGA 800x600 pixeles [iluminación media - día]
- **Dataset_Custom_3:** 56 Fotos sintéticas generadas con GEMINI PRO NANO BANANA a resoluciones variables [espacios libres + variabilidad de escenarios e iluminación]
- **Dataset_Custom_4:** 32 Fotos sintéticas generadas con GEMINI PRO NANO BANANA a resoluciones variables [espacios libres + variabilidad de escenarios e iluminación]
- **Dataset_Custom_5:** 213 Fotos tomadas desde ESP32S3 + CAM OV5640 a resolución FRAMSIZE_SVGA 800x600 pixeles [Variabilidad de escenarios reales] Sin preprocesados ni aumentaciones.

El proceso realizado es:

1. Descomprime el ZIP del dataset original.
2. Lee data.yaml de cada dataset → extrae clases locales.
3. Normaliza nombres de clases (lowercase + singular + excepciones).
4. Construye mapeo global unificado de clases.
5. Mueve imágenes y reescribe labels con el nuevo class_id.
6. Genera data.yaml unificado + log de trazabilidad + análisis de distribución.

Para lo cual se usan funciones del script `Unzip_datasets.py` y `Union_dataset.py` y se entrega el dataset curado en la carpeta `./01_ING_DATOS/Datasets/dataset_IODC_original`.

```
In [2]: # Ruta de carpeta para descomprimir el dataset
dataset_path = os.path.join(base_dir, "Datasets", "dataset_IODC_origin

# Descomprimir el Dataset
unzip_datasets(source_dir=dataset_path)
```

Se encontraron 1 archivos .zip

Descomprimiendo: IndoorObjectDetectionCustom-version-original-yolo26.zip
Destino: IndoorObjectDetectionCustom-version-original-yolo26/
✓ Completado - 2151 archivos extraídos

Proceso completado

```
In [3]: # Ruta del dataset descomprimido (con data.yaml + train/images + train
dataset_path_unzip = os.path.join(dataset_path, "IndoorObjectDetection

# 1. Procesar dataset YOLO: normalizar clases, serializar imágenes, re
stats = process_yolo_dataset(
    dataset_path=dataset_path_unzip,
    output_dir=dataset_path,
    class_overrides={"escalator": "stair"},
    output_dir_reports=output_dir
)
```

🔧 Procesamiento de Dataset YOLO

📁 Origen: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original/IndoorObjectDetectionCustom-version-original-yolo26

📁 Destino: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original

📁 Fase 1: Descubrimiento de clases...

📄 Mapeo global unificado:

0: dog
1: door
2: obstacle
3: person
4: stair

Total de clases únicas: 5

📊 Traducción de clases:

0:'dog' → 0:'dog' (sin cambio)
1:'door' → 1:'door' (sin cambio)

2:'obstacle' → 2:'obstacle' (sin cambio)
3:'person' → 3:'person' (sin cambio)
4:'stair' → 4:'stair' (sin cambio)

📁 Fase 2: Creando estructura de destino...

✅ /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original/data/images

✅ /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original/data/labels

🚗 Fase 3: Procesando imágenes y labels...

Splits encontrados: ['train']

✅ train: 1074 imágenes procesadas

📊 Total de imágenes procesadas: 1074

📄 Fase 4: Generando data.yaml unificado...

✅ /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original/data.yaml

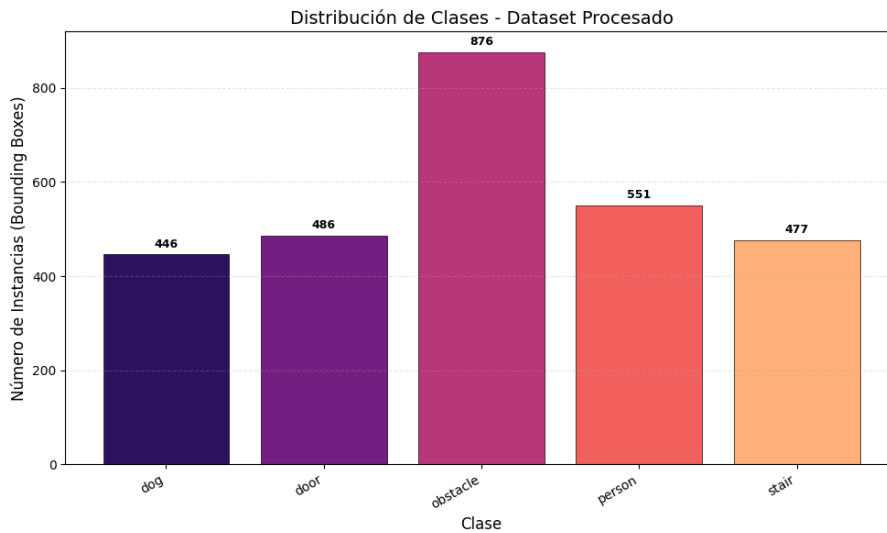
Clases (5): ['dog', 'door', 'obstacle', 'person', 'stair']

📄 Fase 5: Generando log de trazabilidad...

✅ /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original/process_log.csv (1074 registros)

📊 Fase 6: Distribución de clases...

Clase	ID	Instancias	Porcentaje
dog	0	446	15.7%
door	1	486	17.1%
obstacle	2	876	30.9%
person	3	551	19.4%
stair	4	477	16.8%
TOTAL		2,836	100.0%



Gráfica guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/class_distribution.png

✓ PROCESAMIENTO COMPLETADO

Directorio: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets/dataset_IODC_original
 Imágenes: 1,074
 Clases: 5 → ['dog', 'door', 'obstacle', 'person', 'stair']
 BB totales: 2,836

Paso 2: Análisis Técnico del Dataset

Antes de analizar el contenido semántico del dataset, es fundamental verificar la calidad técnica de los datos a nivel de archivos e imágenes.

Dataset: dataset_maestro_unificado (dataset unificado, sin división en splits train/valid/test).

Objetivos:

1. **Analizar resoluciones:** Verificar consistencia de dimensiones de imágenes.
2. **Sanity checks de integridad:**
 - Archivos corruptos o ilegibles
 - Formatos incorrectos (RGBA, grayscale cuando se espera RGB)
 - Duplicados exactos dentro del dataset (contamination)
3. **Estadísticas de píxeles:** Calcular **mean y std por canal RGB** para normalización del modelo.
4. **Distribución de intensidad:** Analizar histogramas RGB para identificar necesidades de augmentation (brightness, contrast).

Impacto en el entrenamiento:

- Imágenes con resoluciones inconsistentes requieren redimensionamiento uniforme.
- Duplicados pueden causar overfitting artificial.
- Channel statistics son **críticas** para la normalización en PyTorch/TensorFlow.

```
In [4]: # Cargar dataset YOLO como objeto COCO
coco, img_dir = load_yolo_as_coco(dataset_path)
splits = {'all': coco}
img_dir_map = {'all': img_dir}
```

```
print(f"✅ Dataset unificado cargado: {len(coco.getImgIds())} imágenes

# Variables de interés para análisis de geometría y complejidad
target_cat_ids = coco.getCatIds()
id_to_name = {cat['id']: cat['name']} for cat in coco.loadCats(target_cat_ids)
print(f"📁 Clases encontradas: {list(id_to_name.values())}")

# Asignar colores automáticamente a cada clase
_color_pool = [
    (255, 0, 0),      # Rojo
    (0, 255, 0),      # Verde
    (0, 0, 255),      # Azul
    (255, 255, 0),    # Cyan/Amarillo
    (255, 0, 255),    # Magenta
    (0, 255, 255),    # Amarillo (BGR)
]
class_colors = {name: _color_pool[i % len(_color_pool)] for i, name in
print(f"🎨 Colores asignados: {class_colors}")
```

loading annotations into memory...

Done (t=0.01s)

creating index...

index created!

✅ Dataset YOLO cargado como COCO:

📁 Imágenes: 1074

📄 Anotaciones: 2836

📁 Categorías: ['dog', 'door', 'obstacle', 'person', 'stair']

✅ Dataset unificado cargado: 1074 imágenes, 2836 anotaciones

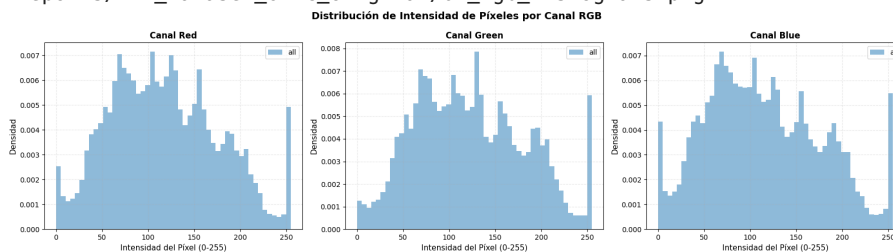
📁 Clases encontradas: ['dog', 'door', 'obstacle', 'person', 'stair']

🎨 Colores asignados: {'dog': (255, 0, 0), 'door': (0, 255, 0), 'obstacle': (0, 0, 255), 'person': (255, 255, 0), 'stair': (255, 0, 255)}

In [5]: # 2. Visualizar distribución de intensidad de píxeles por canal RGB
plot_channel_histograms(splits, base_dir, sample_size=100, img_dir_map
output_dir=output_dir, filename='02_rgb_histog

🎨 Generando histogramas de distribución de píxeles...

📁 Histogramas guardados: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/02_rgb_histograms.png



In [6]: # 3. Calcular estadísticas de canales RGB para normalización del model
df_pixel_stats = calculate_pixel_statistics(splits, base_dir, sample_s
img_dir_map=img_dir_map,
output_dir=output_dir, fil

📊 Calculando estadísticas de canales RGB...

ALL: RGB mean=[0.4536, 0.4768, 0.4409], std=[0.2370, 0.2404, 0.2495]

📁 Estadísticas guardadas: /Users/admin/Documents/TFM_UNIR/01_ING_DATO S/Reports/EDA_Dataset_IODC_original/03_channel_stats.csv

In [7]: # 4. Detectar imágenes duplicadas dentro del dataset usando hash MD5
df_duplicates = detect_duplicates_across_splits(splits, base_dir, img_
output_dir=output_dir,

display(df_duplicates.head())

Visualizar muestra de imágenes duplicadas

if len(df_duplicates) > 0:

dup_img_ids = df_duplicates['image_id'].unique().tolist()

sample_size = min(8, len(dup_img_ids))

sample_dup = random.sample(dup_img_ids, sample_size)

print(f"🔍 Mostrando {sample_size} imágenes duplicadas de {len(dup

```

visualize_sample(coco, target_cat_ids, id_to_name, class_colors, 1
img_ids=sample_dup, output_dir=output_dir, filename
else:
print("✅ No hay imágenes duplicadas para visualizar")

```

🔍 Calculando hashes MD5 de todas las imágenes...

✅ No se encontraron duplicados entre splits

✅ No hay imágenes duplicadas para visualizar

```

In [8]: # 5. Verificar integridad de archivos de imagen
df_integrity = check_file_integrity(splits, base_dir, img_dir_map=img_
output_dir=output_dir, filename='0

```

✅ Verificación de Integridad: SIN PROBLEMAS
Todas las imágenes son válidas y legibles (RGB)

```

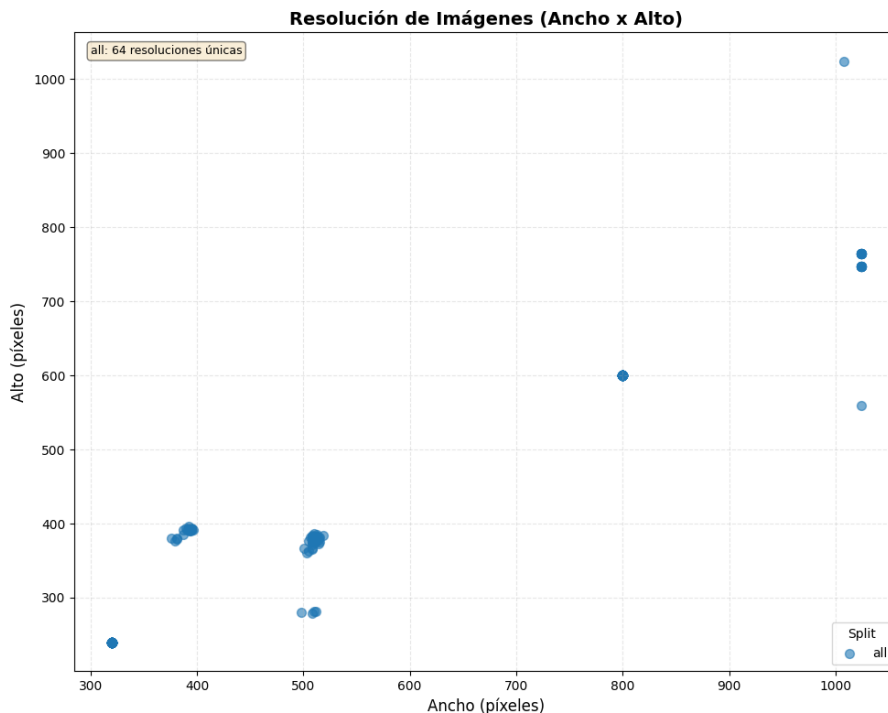
In [9]: # 6. Extraer dimensiones de todas las imágenes
df_dimensions = get_image_dimensions(splits)
plot_resolution_scatter(df_dimensions, output_dir=output_dir, filename

```

/Users/admin/Documents/TFM_UNIR/01_ING_DATOS/src/utils_eda.py:1680: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

unique_resolutions = df_dims.groupby('split').apply(
Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/06_resolution_scatter.png

Tabla de resoluciones guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/06_resolution_scatter_table.csv



Paso 3: Análisis de Distribución de Clases

En esta etapa se decide qué clases se conservan basándose en su representatividad y relevancia estadística.

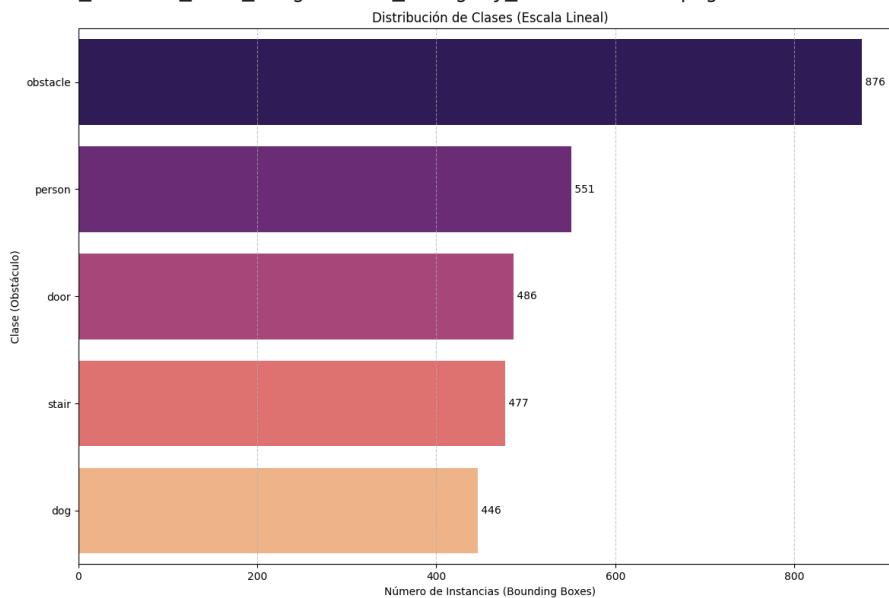
- **Conteo de Instancias por Categoría:**
 - **Objetivo:** Identificar clases "minoritarias" que el modelo no aprenderá bien.

- **Imágenes por Categoría vs. Objetos por Categoría:**
 - **Objetivo:** Determinar si una clase está bien distribuida en el dataset o si aparece concentrada en pocas imágenes.
- **Matriz de Co-ocurrencia:**
 - **Objetivo:** Analizar qué objetos aparecen juntos para entender el contexto (ej. la relación entre "Mesa" y "Silla").

```
In [10]: # Usar dataset unificado completo para análisis de clases
print(f"✅ Usando dataset unificado para análisis de clases: {len(coco)} imágenes")
✅ Usando dataset unificado para análisis de clases: 1074 imágenes
```

```
In [12]: # 7. Obtener categorías y conteos
cat_ids = [cat['id'] for cat in coco.loadCats(coco.getCatIds())]
df_counts = get_category_distribution(coco)
plot_category_distribution(df_counts, scale_type='linear', palette='magma',
                          output_dir=output_dir, filename='07_category')
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_OIDC_original/07_category_distribution.png



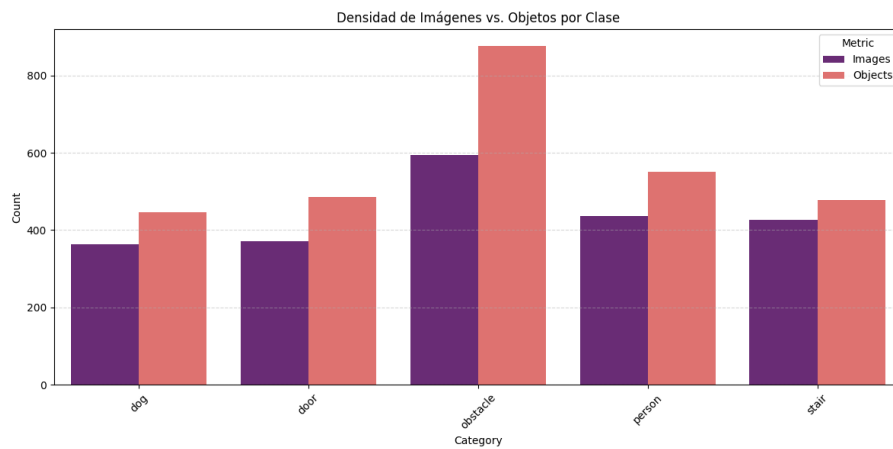
--- Tabla de Distribución ---

Category	Count
obstacle	876
person	551
door	486
stair	477
dog	446

Tabla guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_OIDC_original/07_category_distribution.csv

```
In [13]: # 8. Obtener Densidad de Imágenes VS. Objetos
df_density, active_cat_ids = calculate_image_object_density(coco, cat_ids)
plot_image_object_density(df_density, palette='magma',
                          output_dir=output_dir, filename='08_image_obj')
```

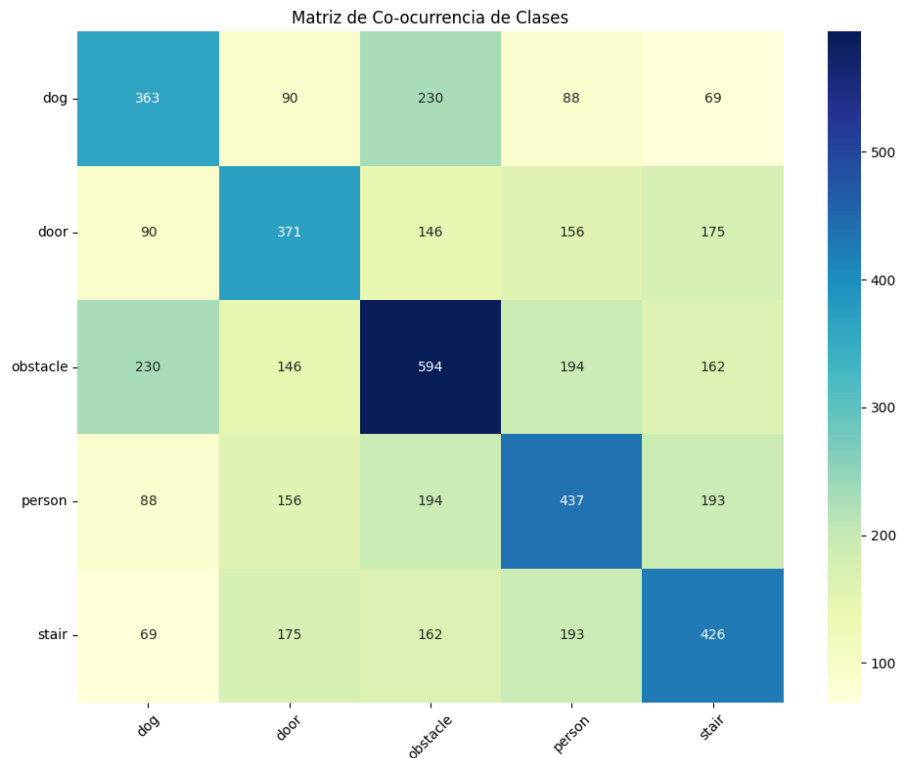
Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_OIDC_original/08_image_object_density.png



In [14]: `# 9. Obtener Matriz de Co-ocurrencia`
`co_occurrence_matrix, cat_labels = calculate_cooccurrence_matrix(coco,`
`plot_cooccurrence_matrix(co_occurrence_matrix, cat_labels, cmap='YlGnB`
`output_dir=output_dir, filename='09_cooccurren`

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report
s/EDA_Dataset_OIDC_original/09_cooccurrence_matrix.png

Matriz guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report
s/EDA_Dataset_OIDC_original/09_cooccurrence_matrix.csv



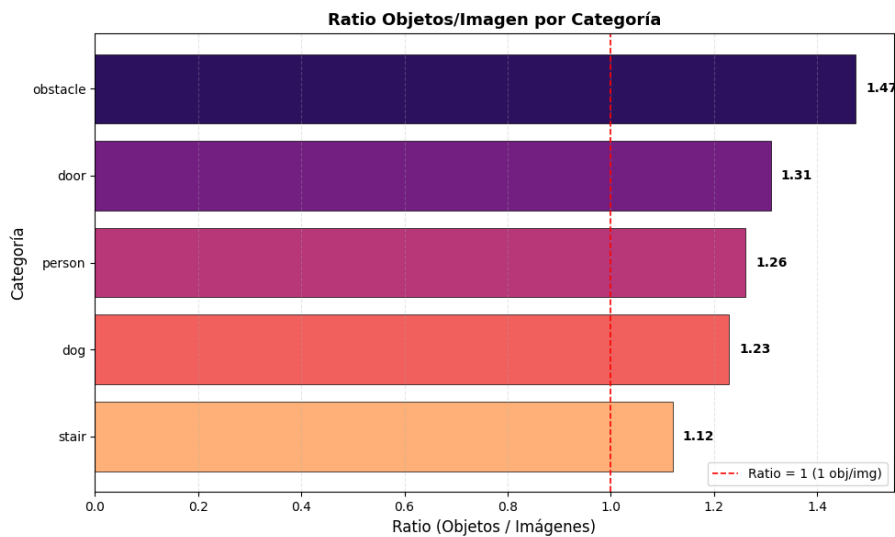
In [15]: `# 10. Calcular ratio Objetos/Imagen por categoría`
`df_ratio = calculate_and_plot_density_ratio(df_density, palette='magma`
`output_dir=output_dir, fil`

--- Ratio de Densidad (Promedio de objetos por imagen donde aparece la
clase) ---

Category	Images	Objects	Ratio
obstacle	594	876	1.4747
door	371	486	1.31
person	437	551	1.2609
dog	363	446	1.2287
stair	426	477	1.1197

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/10_density_ratio.png

Tabla guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/10_density_ratio.csv



Paso 4: Análisis Geométrico de Bounding Boxes

- **Distribución de Tamaños (Área):**
 - **Objetivo:** Clasificar objetos en **Pequeños, Medianos y Grandes** (Estándar COCO).
- **Relación de Aspecto (Aspect Ratio):**
 - **Objetivo:** Graficar ancho vs. alto.
- **Posición Espacial (Heatmap):**
 - **Objetivo:** Mapa de calor de los centros de las cajas.
- **Optimización de Anchors:**
 - **Objetivo:** Calcular anchors óptimos usando K-Means clustering sobre dimensiones de bboxes.

```
In [16]: # 11. Extraer geometría usando función modular
df_geo = extract_bbox_geometry(coco, target_cat_ids,
                               output_dir=output_dir, filename='11_bbo
```

Geometría de Bounding Boxes extraída: 2836 anotaciones

Category	Count	Area_mean	Area_median	Ratio_mean	Ratio_std
dog	446	36792	9916.12	1.05	0.48
door	486	56075.8	29697.9	0.43	0.2
obstacle	876	38609.4	18918.9	0.74	0.37
person	551	45616.3	29704.7	0.49	0.24
stair	477	56647.8	34190.6	1.22	0.52

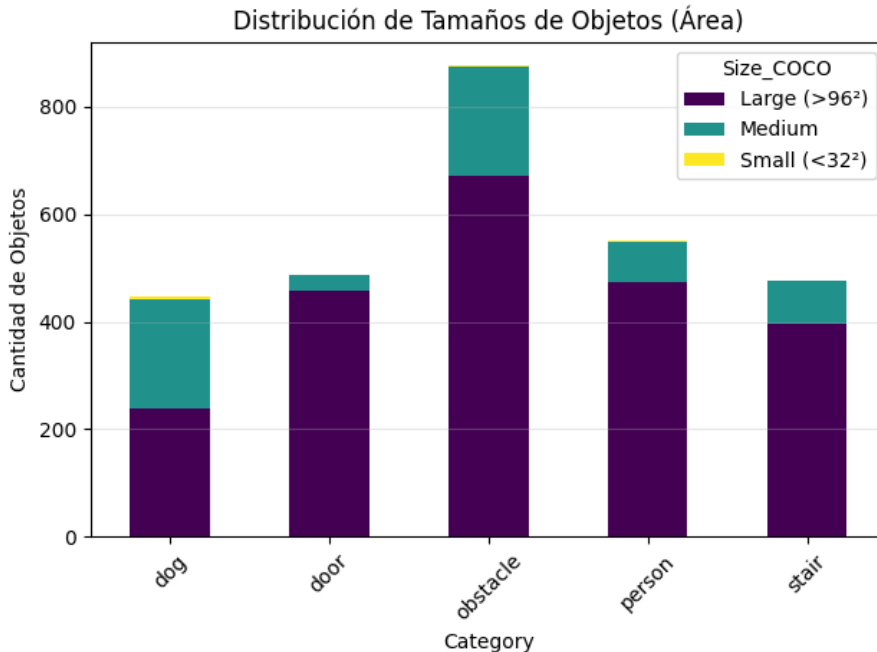
Geometría guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_OIDC_original/11_bbox_geometry.csv

Resumen guardado: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_OIDC_original/11_bbox_geometry_summary.csv

```
In [17]: # 12. Visualizar la Distribución de Tamaños COCO
plot_size_distribution(df_geo, colormap='viridis',
                      output_dir=output_dir, filename='12_size_distribu
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_OIDC_original/12_size_distribution.png

<Figure size 1000x600 with 0 Axes>



--- Porcentaje de Tamaños por Clase ---

Category	Large (>96²)	Medium	Small (<32²)
dog	53.36	45.74	0.9
door	94.03	5.97	0
obstacle	76.71	23.17	0.11
person	85.84	13.79	0.36
stair	83.02	16.98	0

Tabla guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_OIDC_original/12_size_distribution_percentages.csv

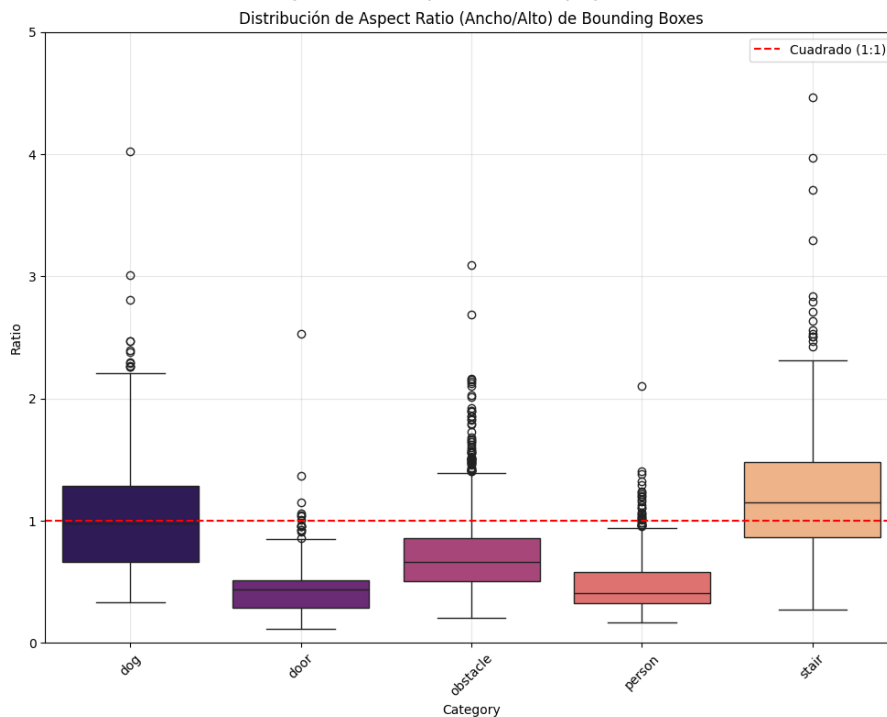
```
In [39]: # 13. Visualizar el Aspect Ratio
plot_aspect_ratio(df_geo, palette='magma',
                  output_dir=output_dir, filename='13_aspect_ratio', fig
```

/Users/admin/Documents/TFM_UNIR/01_ING_DATOS/src/utils_eda.py:1018: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
plt.title(title)
```

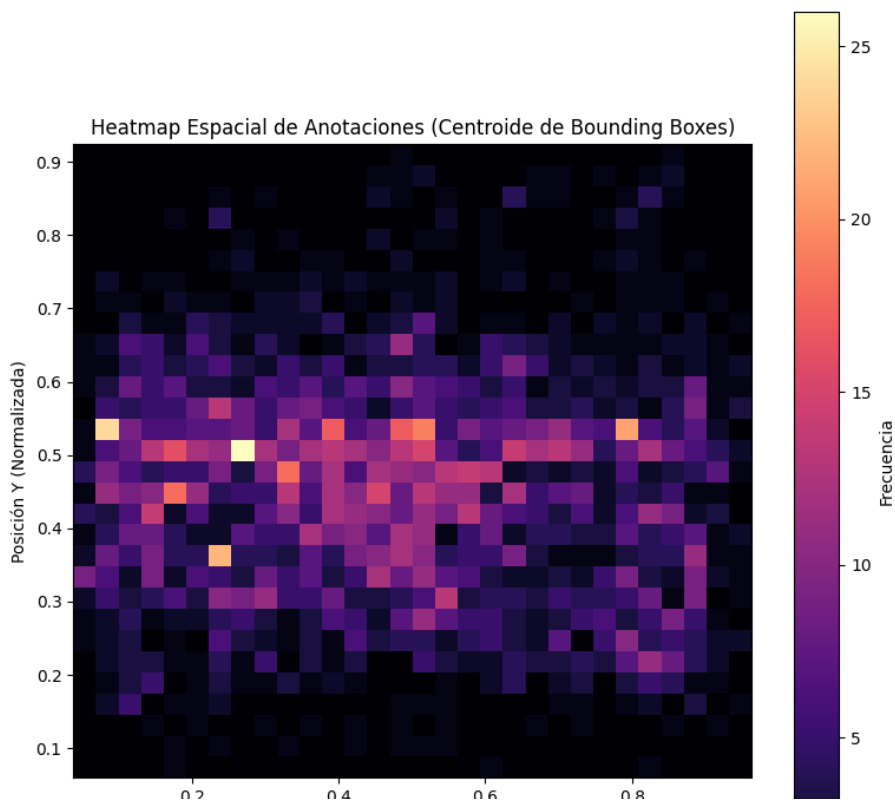
Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_0IDC_original/13_aspect_ratio.png



In [19]:

```
# 14. Visualizar el Heatmap Espacial
plot_spatial_heatmap(df_geo, cmap='magma',
                     output_dir=output_dir, filename='14_spatial_heatmap
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_0IDC_original/14_spatial_heatmap.png



Posición X (Normalizada)

0

```
In [20]: # 15. Extraer dimensiones normalizadas de todos los bboxes
bbox_dims = extract_bbox_dimensions(coco, target_cat_ids)
```

```
print(f"\n📦 Total de bounding boxes extraídos: {len(bbox_dims)}")
print(f"   Dimensiones: ancho y alto normalizados [0,1]")
```

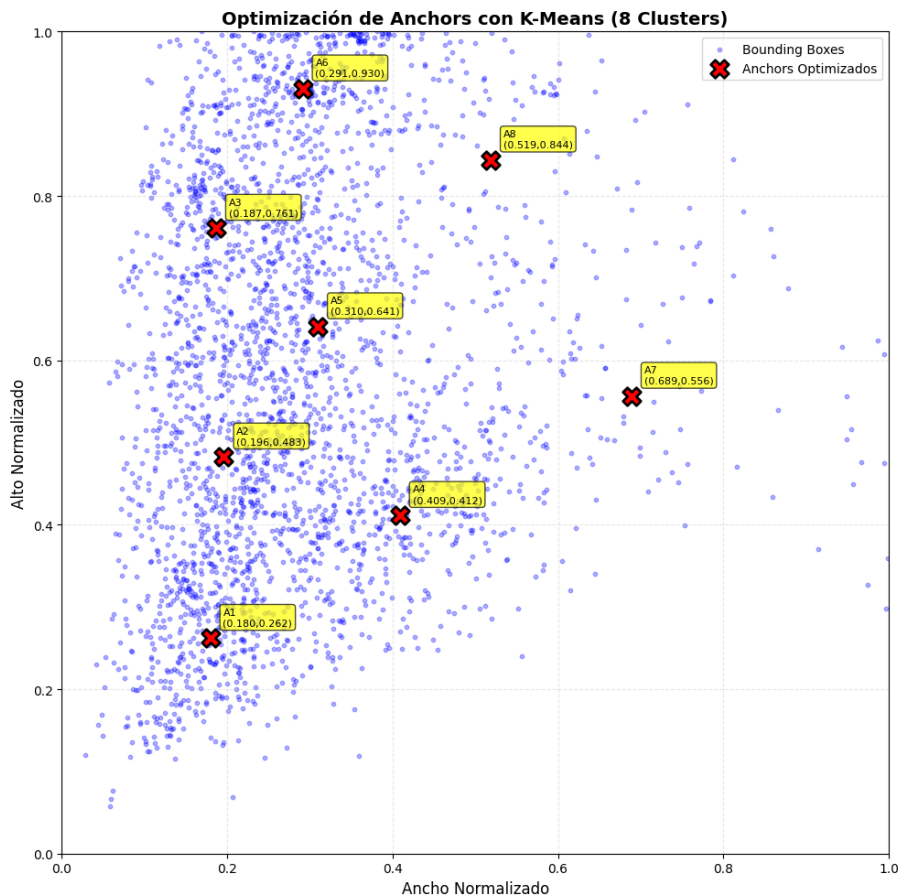
```
📦 Total de bounding boxes extraídos: 2836
   Dimensiones: ancho y alto normalizados [0,1]
```

```
In [41]: # 16. Calcular y visualizar anchors óptimos usando K-Means
optimal_anchors = kmeans_anchor_optimization(bbox_dims, n_anchors=8, r
plot_anchor_clusters(bbox_dims, optimal_anchors, output_dir=output_dir
filename='16_anchor_clusters', title='Optimización
```

```
🔧 Ejecutando K-Means con 8 clusters sobre 2836 bboxes...
```

```
✅ Anchors optimizados calculados
```

```
📁 Visualización de anchors guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/16_anchor_clusters.png
```



```
In [22]: # 17. Exportar configuración YAML con Anchors optimizados
export_yolo_anchors(optimal_anchors, img_size=640, output_dir=output_d
filename='17_yolo_anchors')
```

```
🔧 Configuración de Anchors para YOLO:
```

```
Imagen: 640x640
```

```
Anchors P3/8 (pequeños): [[106, 142], [116, 247], [138, 355]]
```

```
Anchors P4/16 (medianos): [[225, 233], [112, 499], [205, 444]]
```

```
Anchors P5/32 (grandes): [[306, 304], [189, 597], [480, 369], [336, 544]]
```

```
📁 Configuración YAML guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/17_yolo_anchors.yaml
```

```
Usar en modelo: copiar anchors a tu archivo de configuración YOLO
```

Paso 5: Complejidad de la Escena

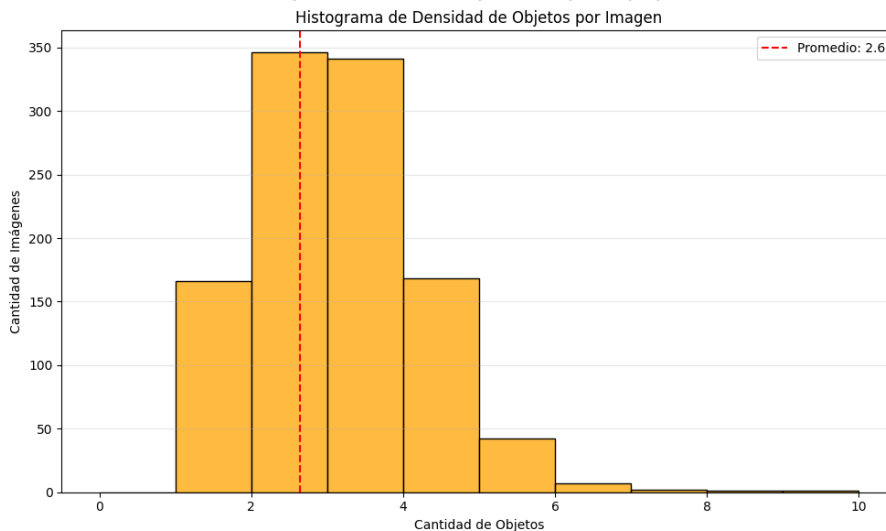
Determina la carga computacional y la dificultad técnica del entrenamiento.

- **Objetos por Imagen (Densidad):**
 - **Objetivo:** Histograma de cantidad de objetos por foto.
 - **Impacto:** Imágenes vacías ayudan a reducir falsos positivos; imágenes muy densas requieren más memoria GPU.
- **Superposición (Oclusión):**
 - **Objetivo:** Calcular el **IoU (Intersection over Union)** entre cajas.
 - **Impacto:** Si hay mucha superposición, se requiere un ajuste fino del *Non-Maximum Suppression (NMS)*.

```
In [42]: # 17. Visualizar Histograma de Densidad
img_ids = coco.getImgIds()
density_counts, ious = calculate_density_and_iou(coco, target_cat_ids)
plot_density_histogram(density_counts, color='orange',
                      output_dir=output_dir, filename='17_density_histo
```

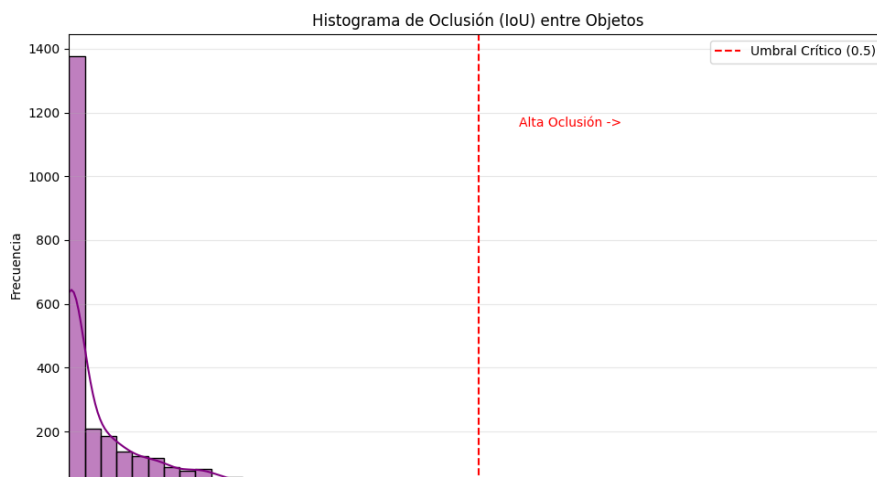
Procesando imágenes para calcular IoU y Densidad...

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_0IDC_original/17_density_histogram.png



```
In [43]: # 18. Visualizar Histograma de Oclusión (IoU)
plot_iou_histogram(ious, color='purple',
                  output_dir=output_dir, filename='18_iou_histogram', t
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_0IDC_original/18_iou_histogram.png





```
In [25]: # 19. Resumen numérico de Densidad
print_density_stats(img_ids, density_counts,
                    output_dir=output_dir, filename='19_density_stats')
```

Total Imágenes Analizadas: 1074

Imágenes Vacías (Background): 0

Promedio de objetos por imagen: 2.64

Máximo de objetos en una imagen: 9

Estadísticas guardadas: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/19_density_stats.csv

Paso 6: Inspección Visual Cualitativa (Sanity Check)

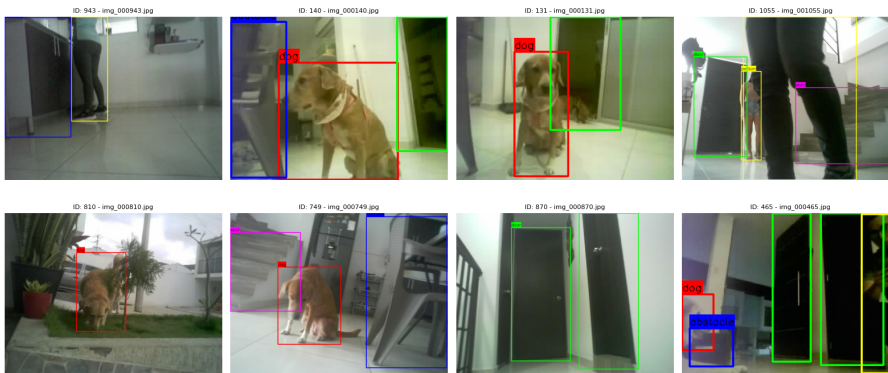
Validación manual para asegurar la calidad de la información.

1. **Visualización Randomizada:** Renderizar imágenes con sus etiquetas para confirmar la coherencia de las clases elegidas.
2. **Verificación de "Ruido":** Identificar errores de etiquetado, como cajas que cubren toda la imagen o que son de tamaño minúsculo.

```
In [26]: # 20. Visualizar Sample de Imágenes
visualize_sample(coco, target_cat_ids, id_to_name, class_colors, img_d
                output_dir=output_dir, filename='20_visual_sample_1', t
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/20_visual_sample_1.png

Muestra #1 de Imágenes con Anotaciones



```
In [27]: # 21. Visualizar Sample de Imágenes
visualize_sample(coco, target_cat_ids, id_to_name, class_colors, img_d
                output_dir=output_dir, filename='21_visual_sample_2', t
```

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Reports/EDA_Dataset_IODC_original/21_visual_sample_2.png

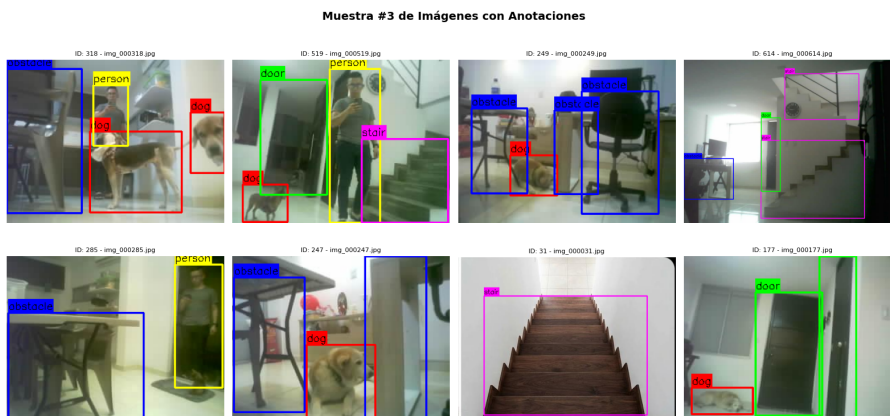
Muestra #2 de Imágenes con Anotaciones





In [28]: `# 22. Visualizar Sample de Imágenes`
`visualize_sample(coco, target_cat_ids, id_to_name, class_colors, img_d`
`output_dir=output_dir, filename='22_visual_sample_3', t`

Figura guardada: /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Report s/EDA_Dataset_IODC_original/22_visual_sample_3.png



In [29]: `# 23. Detectar anomalías`
`suspect_tiny, suspect_huge = detect_anomalies(coco, target_cat_ids, id`
`output_dir=output_dir, fi`

--- Iniciando búsqueda de anomalías ---

Resultados del escaneo:

⚠ Cajas Minúsculas (<15x15px): 0

⚠ Cajas Gigantes (>95.0% imagen): 0

In [30]: `# 24. Visualizar imágenes anómalas de cajas minúsculas`
`if len(suspect_tiny) > 0:`
`print("\n Ejemplos de imágenes anómalas: con cajas minúsculas")`
`unique_tiny_ids = list(set([s['img_id'] for s in suspect_tiny]))`
`sample_size = min(8, len(unique_tiny_ids)) # Máximo 8 imágenes en`
`sample_tiny = random.sample(unique_tiny_ids, sample_size)`

`# Usar grilla de visualize_sample para mostrar múltiples imágenes`
`visualize_sample(coco, target_cat_ids, id_to_name, class_colors, i`
`img_ids=sample_tiny, output_dir=output_dir, filena`

In [31]: `# 25. Visualizar imágenes anómalas con cajas gigantes`
`if len(suspect_huge) > 0:`
`print("\n Ejemplos de imágenes anómalas: con cajas gigantes")`
`unique_huge_ids = list(set([s['img_id'] for s in suspect_huge]))`
`sample_size = min(8, len(unique_huge_ids)) # Máximo 8 imágenes en`
`sample_huge = random.sample(unique_huge_ids, sample_size)`

`# Usar grilla de visualize_sample para mostrar múltiples imágenes`
`visualize_sample(coco, target_cat_ids, id_to_name, class_colors, i`
`img_ids=sample_huge, output_dir=output_dir, filena`

Síntesis del EDA: Dataset Indoor Object

Detection Custom (IODC) Original

La siguiente tabla resume los hallazgos críticos del Análisis Exploratorio de Datos del dataset IODC original (pre-augmentation), sus implicaciones técnicas para un sistema de navegación robótica basado en ESP32-S3 (TinyML) y las acciones correctivas definidas.

Paso del EDA	Tarea Realizada	Hallazgos Clave	Implicaciones Técnicas	Acción a realizar en Preprocesado	A c E
Paso 1	#1. Procesamiento y visualización de la distribución de clases del dataset IODC original	5 clases, 2,836 anotaciones en 1,074 imágenes. Distribución moderadamente equilibrada: obstacle (876, 30.9%) lidera; dog (446, 15.7%) es la clase minoritaria. Ratio max/min = 1.96:1. Solo se encontró split "train" (sin val/test).	El desbalance es leve (ratio <2:1), muy inferior al de datasets combinados. No se espera sesgo severo del modelo. Aun así, obstacle duplica a dog, y para navegación robótica, dog (agente biológico impredecible) requiere recall alto.	Aplicar augmentation uniforme y considerar leve oversampling de dog. Realizar split train/val/test al no existir particiones previas.	N p w ir p fi s d s e
Paso 2	#2. Visualizar distribución de intensidad de píxeles por canal RGB	Distribución unimodal centrada en los 3 canales con picos principales en rango 50–150. Sin bimodalidad extrema. Canal Blue ligeramente inferior. Sin pico pronunciado en 255 (sin sobreexposición sistemática).	La distribución centrada indica escenas de iluminación interior relativamente homogénea. Menor varianza de iluminación que un dataset combinado multi-fuente, lo que podría limitar generalización a condiciones extremas de luz.	Aplicar augmentation de brillo, contraste y gamma para ampliar cobertura del espacio de iluminación y mejorar robustez.	N
Paso 2	#3. Calcular estadísticas de canales RGB para normalización del modelo	Mean RGB = [0.4536, 0.4768, 0.4409], Std RGB = [0.2370, 0.2404, 0.2495]. Media ligeramente inferior a 0.5, std ~0.24 uniforme entre canales. Valores difieren de ImageNet (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) pero son más próximos que el dataset combinado.	La normalización es crítica para convergencia. Si se usa transfer learning con backbone preentrenado en ImageNet, usar estadísticas ImageNet; si se entrena from scratch, usar las propias del dataset.	Almacenar mean/std para el pipeline de normalización.	S (I T c lr s n O I O
Paso 2	#4. Detectar imágenes duplicadas usando hash MD5	✅ 0 imágenes duplicadas detectadas. Dataset completamente libre de duplicados.	Sin riesgo de data leakage por duplicados. El split train/val/test puede realizarse directamente sin deduplicación previa. Excelente calidad de curación del dataset.	NA (no se requiere deduplicación).	N
	#5. Verificar	✅ Todas las 1,074 imágenes son válidas, legibles y en	No se requiere limpieza de		

Paso 2	#5. Verificar integridad de archivos de imagen	validas, legibles y en formato RGB. Sin archivos corruptos, truncados ni en formatos inesperados.	limpieza de archivos. El pipeline de carga no necesita manejo especial de errores.	NA	N
Paso 2	#6. Extraer dimensiones de todas las imágenes	64 resoluciones únicas encontradas. Resoluciones variables: desde 320×240 hasta 1024×1024, con concentraciones en ~380×380, ~500×370, 640×480, 800×600 y 1024×750. No hay resolución uniforme.	A diferencia de datasets pre-procesados a 640×640, este dataset original tiene resoluciones heterogéneas. Todas las imágenes necesitarán redimensionamiento para crear batches uniformes. La variedad de resoluciones es positiva para robustez del modelo (entrena con múltiples escalas implícitas).	Crítico: Redimensionar todas las imágenes a resolución uniforme (640×640 para YOLO, o 192×192 para TinyML directo). Decidir estrategia: letterbox (preserva aspect ratio) vs resize directo.	C e n a d P p ri (i p b
Paso 3	#7. Obtener categorías y conteos (distribución detallada)	Confirma distribución: obstacle (876), person (551), door (486), stair (477), dog (446). Las 3 clases top (obstacle, person, door) acumulan 67.5% de las instancias. Las 5 clases están en rango 15.7%–30.9%.	Desbalance bajo. Todas las clases tienen representación suficiente (>400 instancias) para entrenamiento efectivo. La distribución es significativamente más equilibrada que datasets combinados típicos.	Augmentation uniforme; no se requiere augmentation fuertemente diferenciada por clase.	C o L n b p e
Paso 3	#8. Obtener Densidad de Imágenes vs. Objetos	obstacle: 594 img / 876 obj. person: 437 img / 551 obj. stair: 426 img / 477 obj. door: 371 img / 486 obj. dog: 363 img / 446 obj. Todas las clases aparecen en >33% de las imágenes.	Distribución homogénea: las 5 clases están presentes en un rango similar de imágenes (363–594). No hay clases "raras" confinadas a pocas imágenes. Esto favorece aprendizaje uniforme de todas las clases.	NA	N
Paso 3	#9. Obtener Matriz de Co-ocurrencia	Co-ocurrencia significativa entre clases: obstacle-dog (230 img), obstacle-person (194 img), person-stair (193 img), door-stair (175 img), door-person (156 img). dog es la clase con menor co-ocurrencia con otras (69–230 img).	A diferencia de datasets multi-fuente con clases independientes, este dataset muestra escenas naturales multi-clase. El modelo aprenderá contextos realistas de co-ocurrencia (e.g., persona+puerta, obstáculo+perro). Favorable para detección contextual.	NA	L n a c a ri e c
			Escenas con baja densidad por clase: típicamente 1		n

Paso 3	#10. Calcular ratio Objetos/Imagen por categoría	obstacle: 1.47 obj/img, door: 1.31, person: 1.26, dog: 1.23, stair: 1.12. Ratios uniformemente bajos (1.1–1.5).	instancia por clase por imagen. Favorable para hardware limitado (ESP32-S3). NMS será simple al no haber muchas instancias solapadas de la misma clase.	NA	" p c 1 r: tl (- s
Paso 4	#11. Extraer geometría de bounding boxes	2,836 anotaciones analizadas. stair: mayor área media (56,648 px²), door (56,076). dog es la clase con mayor variabilidad (area_mean=36,792, area_median=9,916, ratio ~4:1 entre media y mediana). Aspect ratio: door (0.43, vertical), person (0.49, vertical), obstacle (0.74), dog (1.05, cuadrado), stair (1.22, horizontal).	Gran diversidad de formas. door y person son claramente verticales; stair es horizontal; dog y obstacle son más variables. La discrepancia media/mediana en dog indica distribución sesgada con outliers de área grande.	NA	L a (' Y K o h g f a F
Paso 4	#12. Visualizar la Distribución de Tamaños COCO	Predominancia de objetos Large (>96²): door 94.0%, person 85.8%, stair 83.0%, obstacle 76.7%, dog 53.4%. dog tiene la mayor proporción de Medium (45.7%). Small objects son marginales (<1% en todas las clases).	Dataset dominado por objetos medianos y grandes, coherente con cámaras indoor a corta distancia. dog es la clase más variable en tamaño (casi mitad Large/Medium). Para ESP32-S3 a 192x192, objetos grandes seguirán siendo detectables; los pocos Small se perderán.	NA	P P n P s a T r e d b ti
Paso 4	#13. Visualizar el Aspect Ratio	door y person tienden a ser verticales (mediana <0.5). dog tiene dispersión moderada (IQR ~0.6–1.3). stair es horizontal con alta dispersión y outliers hasta 4.5. obstacle centrado ~0.7 con outliers hasta 3.1.	La diversidad de aspect ratios confirma necesidad de anchors variados o modelo anchor-free. Para TinyML, la cuantización debe preservar capacidad de detectar formas verticales (puertas, personas) y horizontales (escaleras).	NA	A r: s fi e s e a
Paso 4	#14. Visualizar el Heatmap Espacial	Concentración de centroides en banda horizontal central (Y: 0.3–0.6), con ligero sesgo hacia la mitad izquierda. Algunos hot spots puntuales en la zona superior-izquierda. Distribución más dispersa que un dataset centrado artificialmente.	La concentración en la franja central-horizontal es coherente con cámara a altura de robot (objetos en el horizonte). El sesgo leve hacia la izquierda podría indicar composición fotográfica sistemática, pero no es severo.	NA	N

Paso 4	#15. Extraer dimensiones normalizadas de todos los bboxes	2,836 bounding boxes extraídos con ancho y alto normalizados en rango [0,1]. Paso intermedio para optimización de anchors.	Normalización permite comparar dimensiones independientemente de la resolución (importante dado que este dataset tiene 64 resoluciones distintas). Facilita cálculo de anchors transferibles.	NA	N
Paso 4	#16. Calcular y visualizar anchors óptimos usando K-Means (8 Clusters)	8 anchors calculados sobre 2,836 bboxes. Rango desde A1 (0.180×0.262) hasta A8 (0.519×0.844). Distribución cubre bien el espacio de dimensiones con clusters de pequeño a grande. Concentración de anchors en zona media (0.2–0.5 ancho).	Los anchors personalizados capturan la distribución específica de este dataset. Relevante solo para modelos anchor-based (YOLOv5, v7). Los 8 clusters cubren adecuadamente las 5 clases con sus diferentes geometrías.	NA	S b e Y (i ir c d t:
Paso 4	#17a. Exportar configuración YAML con Anchors optimizados para 640×640	Anchors exportados en 3 escalas: P3/8 (pequeños): [106×142], [116×247], [138×355]; P4/16 (medianos): [225×233], [112×499], [205×444]; P5/32 (grandes): [306×304], [189×597], [480×369], [336×544].	YAML listo para configuración YOLO. Anchors cubren desde ~17% hasta ~75% del tamaño de imagen 640×640. Para TinyML, reescalar proporcionalmente a resolución de inferencia.	NA	C Y b r
Paso 5	#17b. Visualizar Histograma de Densidad de objetos por imagen	Distribución centrada con promedio de 2.6 obj/img. Mayoría de imágenes tiene 2–4 objetos. Máximo: ~10 objetos por imagen. Sin outliers extremos. Sin escala logarítmica	Escenas de complejidad moderada y uniforme (2–4 objetos). Sin outliers extremos (máx ~10 vs 96 del dataset combinado). Favorable para ESP32-S3: carga de	NA	n 1 B e fi p n
Paso 5	#18. Visualizar Histograma de Oclusión (IoU) entre Objetos	necesaria (distribución compacta). Mayoría de pares con IoU bajo (<0.1), distribución exponencialmente decreciente. Muy pocos casos con IoU >0.3. Prácticamente nulos sobre 0.5 (umbral crítico de alta oclusión).	detección predecible y acotada. Baja oclusión generalizada. NMS simple será suficiente en todas las escenas. No se observan pares con IoU ≈ 1.0 (sin anotaciones duplicadas superpuestas, a diferencia del dataset combinado).	NA	N e O N s v s

Paso 5	#19. Resumen numérico de Densidad	1,074 imágenes analizadas. 0 imágenes vacías (background). Promedio: 2.64 obj/img. Máximo: 9 obj/img.	Sin imágenes background (vacías). Todas las imágenes contienen al menos un objeto. Para reducir falsos positivos, se necesitará generar o incorporar imágenes background artificialmente. El promedio bajo (~2.6 obj/img) es favorable para hardware limitado.	Considerar añadir imágenes negativas (sin objetos) al dataset para reducir falsos positivos del modelo.	Ir s e n n " d e
Paso 6	#20, #21, #22. Visualizar 3 muestras aleatorias de imágenes con anotaciones (Sanity Check)	Las muestras muestran anotaciones visualmente coherentes con bounding boxes bien ajustados. Diversidad de escenarios interiores naturales: habitaciones, pasillos, escaleras, puertas, obstáculos variados. Imágenes con resoluciones variadas confirmando la heterogeneidad dimensional.	Calidad general de anotación aceptable. Las imágenes provienen de escenas interiores reales (no catálogo/e-commerce), lo que favorece la representatividad para el caso de uso del robot navegador.	NA	N
Paso 6	#23. Detectar anomalías en anotaciones	✅ 0 cajas minúsculas (<15x15px) detectadas. ✅ 0 cajas gigantes (>95% del frame) detectadas. Dataset completamente limpio de anomalías en anotaciones.	Excelente calidad de anotación: sin artefactos de borde, sin bboxes degenerados, sin anotaciones que cubran el frame completo. No se requiere limpieza de anotaciones.	NA (dataset limpio).	N
Paso 6	#24, #25. Visualizar imágenes anómalas	Sin imágenes anómalas para visualizar (0 tiny, 0 huge).	NA	NA	N

🎯 Conclusión Final:

El dataset IODC original presenta una base de **1,074 imágenes** con resoluciones heterogéneas (64 resoluciones únicas) y **2,836 anotaciones** en **5 clases** relevantes para navegación robótica indoor (obstacle, door, person, stair, dog). La calidad del dataset es alta: sin duplicados, sin archivos corruptos, sin anomalías de anotación, y con distribución de clases moderadamente equilibrada (ratio 1.96:1). Se identifican **3 acciones de preprocesado** antes de entrenar:

- 1. Redimensionamiento uniforme (Prioridad ALTA):** Las 64 resoluciones diferentes deben normalizarse a 640x640 (letterbox o resize) para crear batches de entrenamiento uniformes.
- 2. Split train/val/test (Prioridad ALTA):** El dataset solo tiene split "train"; es necesario crear particiones val/test estratificadas.
- 3. Incorporar imágenes background:** El dataset no contiene imágenes vacías (0 background). Añadir negative samples para reducir falsos positivos en navegación real.

Para el **entrenamiento orientado a TinyML/ESP32-S3**, las principales consideraciones son:

- **Augmentation uniforme** con énfasis en brillo/contraste para compensar la baja variabilidad de iluminación del dataset de fuente única.
- **max_detections ≤ 15** es suficiente dado el máximo de 9 objetos por imagen.
- Priorizar **cabezas P4/P5** (objetos medianos/grandes) y simplificar P3 en la arquitectura TinyML.
- Entrenar con **progressive resizing** (640→320→192) para robustez a baja resolución, aprovechando la diversidad nativa de resoluciones.
- **Class weights opcionales**: el desbalance leve (1.96:1) puede no requerir Focal Loss, pero evaluar su impacto en dog.
- **NMS simplificado (greedy)**: viable dado el bajo solapamiento entre objetos (IoU mayoritariamente < 0.1).

Preprocesado del Dataset

Esta sección del notebook se centra en la preparación de los datos para el entrenamiento del modelo de detección de objetos.

- **Paso 1:** Data Augmentation
- **Paso 2:** Split del Dataset (train/valid/test)
- **Paso 3:** Normalización de Imágenes (mean/std)
- **Paso 4:** Guardar el Dataset Preprocesado en Formato YOLO

Paso 1: Data Augmentation (Análisis y Planificación)

Una vez identificadas las clases minoritarias en las fases anteriores, es momento de planificar la estrategia de **Data Augmentation** para balancear el dataset.

Objetivos:

1. **Identificar necesidades:** Calcular déficit de instancias por clase respecto a un objetivo.
2. **Planificar estrategia:** Determinar cuántas copias aumentadas generar por imagen candidata.
3. **Visualizar impacto:** Proyectar distribución ANTES vs DESPUÉS del augmentation.
4. **Ejecutar pipeline:** (Opcional) Integrar transformaciones de albumentations para generar dataset balanceado.

Estrategias de Transformación:

- **Mild (Suave):** HorizontalFlip, RandomBrightnessContrast, GaussNoise, GaussianBlur - para clases con déficit < 100 instancias.
- **Aggressive (Agresiva):** Affine (rotación, escala, traslación), ColorJitter, RandomGamma, MotionBlur - para clases altamente minoritarias.

Nota: El pipeline completo de augmentation se ejecuta mediante el script standalone `01_ING_DATOS/Datasets/Augmentation_dataset.py` que genera un nuevo dataset sin modificar el original.

```
In [32]: # --- ANÁLISIS DE NECESIDADES DE AUGMENTATION ---
# Definir objetivo de instancias por clase (ajustar según criterio)
# TARGET_INSTANCES = 400 # Valor objetivo para balanceo

# Analizar qué clases son minoritarias
# df_augmentation_analysis = analyze_augmentation_needs(coco, target_i
#                                     output_dir=out
#                                     filename='25_a
```

```
In [33]: # --- PLANIFICACIÓN DE ESTRATEGIA ---
# Calcular cuántas copias aumentadas se necesitan por imagen
# df_augmentation_plan = compute_augmentation_strategy(df_augmentation
#                                     output_dir=outp
#                                     filename='26_au
```

```
In [34]: # --- VISUALIZACIÓN DEL IMPACTO ---
# Comparar distribución ANTES vs DESPUÉS (proyectado)
# if len(df_augmentation_plan) > 0:
#     visualize_augmentation_impact(df_augmentation_analysis, df_augmen
#                                     output_dir=output_dir, filename='27
# else:
#     print("\n✅ No se requiere visualización: dataset ya está balance
```

Ejecución del Pipeline de Augmentation

Para ejecutar el pipeline completo de data augmentation con albumentations:

```
cd /Users/admin/Documents/TFM_UNIR/01_ING_DATOS/Datasets
python Augmentation_dataset.py
```

El script realiza:

1. Copia completa del dataset original a `dataset_maestro_aug/`
2. Identifica imágenes candidatas que contienen classes minoritarias
3. Aplica transformaciones mild/aggressive según el déficit
4. Genera imágenes aumentadas con sus anotaciones YOLO
5. Exporta:
 - `augmentation_log.csv` : Trazabilidad de imágenes generadas
 - `class_distribution.png` : Gráfico ANTES/DESPUÉS
 - `data.yaml` : Configuración actualizada

Ventajas:

- ✅ No modifica el dataset original
- ✅ Preserva formato YOLO (imágenes + labels .txt)
- ✅ Transformaciones bbox-aware (no corta objetos)
- ✅ Log completo de trazabilidad

Paso 2: Split de Dataset

- **Comparación Multi-Split:**
 - **Objetivo:** Verificar que train/valid/test tengan distribuciones de clases similares (test Chi-cuadrado).

```
In [35]: # --- COMPARACIÓN DE DISTRIBUCIÓN ENTRE SPLITS ---
# NOTA: No aplica para el dataset unificado (no hay splits train/valid)
# Esta comparación se realizará después de dividir el dataset en split
# print("🚫 Comparación multi-split omitida: el dataset unificado no
```

```
In [36]: # Test Chi-cuadrado para verificar homogeneidad estadística entre spli
# NOTA: No aplica para el dataset unificado (requiere múltiples splits)
# Se ejecutará después de dividir el dataset en splits para entrenamie
# print("🚫 Test Chi-cuadrado omitido: requiere múltiples splits.")
```

```
# print( "Test en el cuadro siguiente, requiere multiples spaces. "
```

```
In [37]: # Analizar distribución de imágenes y anotaciones por split  
# df_split_dist = analyze_split_distribution(splits, output_dir=output
```

Paso 3: Guardar Dataset Preprocesado en Formato YOLO